University of Management and Technology

Fake News Detection :

Submitted by:

Hassan Bahar F2023266769

Hassan Kareem F2023266768

M.Adan F2023266794

Course: Machine Learning

Instructor: Prof.M. Tahir Sohail

Due Date: 02/July/2025

1. Introduction

Fake news detection is a critical problem in modern media. Misinformation spreads rapidly through social platforms, often impacting public opinion and decision-making. In this project, we use machine learning models to classify news articles as fake or real. We use two datasets: one containing fake news and the other real news articles.

Defining Dataset

The dataset contains two CSV files: 'Fake.csv' and 'True.csv'. Each file contains a column with news text. We label fake news with 0 and true news with 1. We combine and shuffle both datasets into a single DataFrame for processing.

Dataset Characteristics

The dataset used in this project contains news articles collected from online sources, divided into two categories: fake and true news. It combines two CSV files, one with fake news and the other with real news. Each article is labeled as either 0 for fake or 1 for true. The dataset has around 44,898 articles in total, with a fairly equal number of fake and true entries, which helps make the model more accurate and unbiased. The main column used is the article text, and additional columns were added for cleaned text and word count. The text data varies in length and style, which makes it suitable for training a machine learning model to detect fake news based on the writing patterns and content. **Real-Life**

Applications:

1. Social Media Monitoring & Fact-Checking

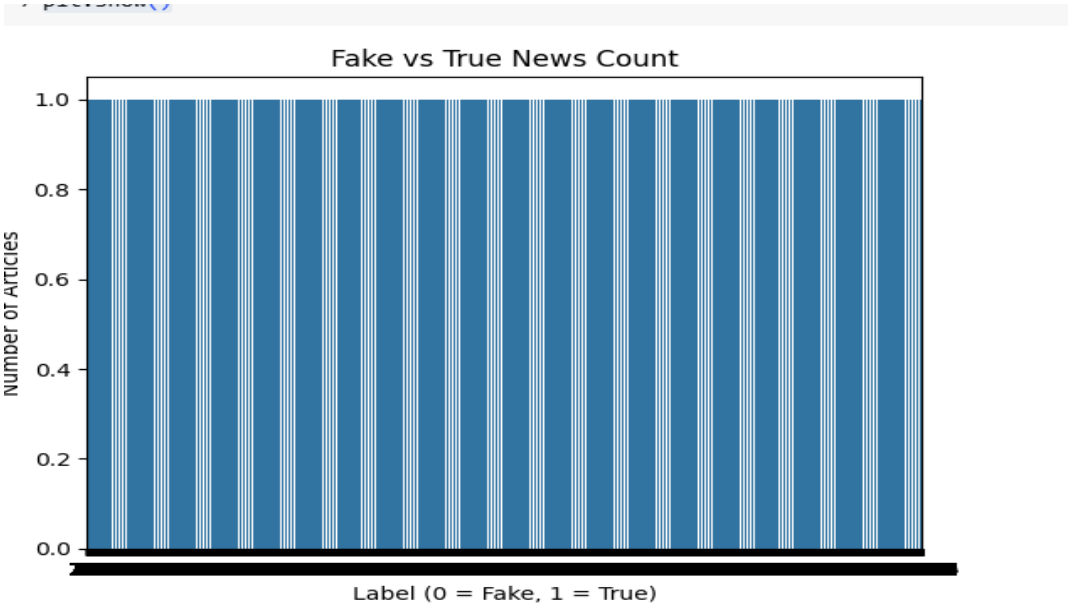
- Platforms like **Facebook, Twitter, YouTube**, and **Instagram** can integrate fake news detection models to **automatically flag or block misleading articles** before they go viral.

2. Integration with Chatbots and Virtual Assistants

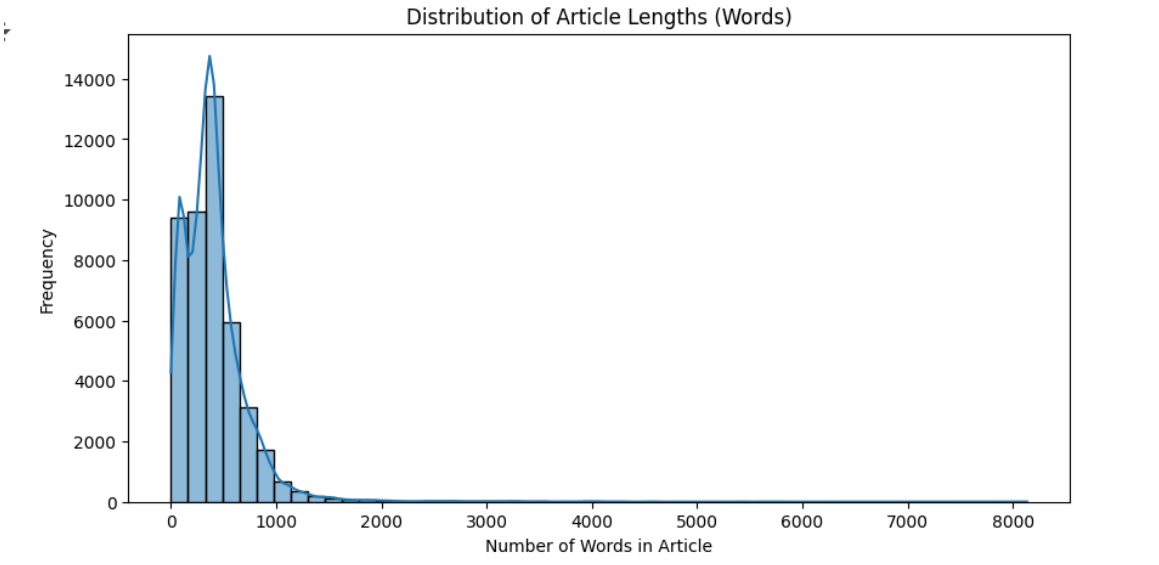
- Assistants like **Siri**, **Google Assistant**, or **Alexa** can use a fake news classifier as a backend filter when providing article recommendations or search results.

Graphs:

Barchart:



Histogram:



2. Methodology

This project follows a step-by-step process to build a machine learning system that can classify news articles as fake or true. Each step was important in improving the accuracy and performance of the model.

1. **Data Collection**:
   We used two CSV files from Kaggle: one for fake news and one for true news. Each article was labeled as 0 (fake) or 1 (true) and both datasets were combined into a single dataset.
2. **Data Preprocessing**:
   We cleaned the news text to remove noise like punctuation, numbers, and special characters. We also:

   - Converted all text to lowercase
   - Removed stopwords like “is”, “the”, “and” using NLTK
   - Tokenized the text into words
   - Rejoined the cleaned words into complete cleaned text

3.Feature Extraction(TFIDF) :

We used TfidfVectorizer to convert the text into numerical features that a machine learning model can understand. This method gives more importance to unique words in each article and less importance to common words.

4.N-gram Techniques:

We applied three n-gram approaches:

- **1-gram (unigrams)**: looks at single words
- **2-gram (bigrams)**: looks at pairs of words like “prime minister”
- **3-gram (trigrams)**: looks at groups of three words

Changing the n-gram range changes the way the model understands text. For example:

- 1-gram is simple but may miss important word combinations.
- 2-gram and 3-gram can understand more context but may include too many features, which can make the model slower or less accurate if the data is small.

That’s why we tested all three to compare their performance.

5.Models Used and Why:

We trained three models:

1. **Logistic Regression** – This is a simple and fast model, good for binary classification tasks like fake vs. true news.
2. **Random Forest Classifier** – A powerful model that uses many decision trees and gives high accuracy. It handles large features (like in text data) very well.
3. **MLPClassifier (Neural Network)** – A model that tries to learn deeper patterns in the data using layers. It is slower but can learn complex relationships in the text.

We used these three to compare traditional machine learning with ensemble learning and a basic neural network.

6.Evaluation Metrics and Their Purpose:

To check how well our models performed, we used the following metrics:

- **Accuracy**: How many predictions were correct overall.
- **Precision**: Out of the news articles marked as real, how many were truly real.
- **Recall**: Out of all the real news articles, how many did the model correctly identify.
- **F1-Score**: A balance between precision and recall.

These metrics are useful because they help us know not just how many answers were correct, but also how careful the model is when labeling fake or true articles. We also used a **confusion matrix** to visualize which articles were misclassified.

3.Results:

In this section, we present the results of our models using different n-gram configurations. Each model was trained three times using 1-gram, 2-gram, and 3-gram. We then compared their performance using evaluation metrics like accuracy, precision, recall, and F1-score. We also used confusion matrices to visually show how well the models predicted fake and true news.

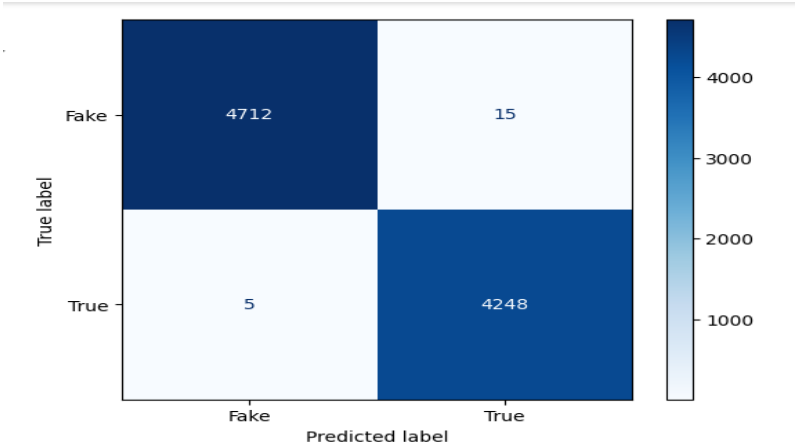
Table:

Evaluation Score Comparison Table

Model	N-Gram	Accuracy	Precision	Recall	F1-Score
Logistic Regression	(1, 1)	0.99	0.99	0.99	0.99
Logistic Regression	(2, 2)	0.98	0.98	0.98	0.98
Logistic Regression	(3, 3)	0.95	0.95	0.95	0.95
Random Forest	(1, 1)	0.9979	1.00	1.00	1.00
Random Forest	(2, 2)	0.9802	0.98	0.98	0.98
Random Forest	(3, 3)	0.9448	0.94	0.94	0.94
MLP Classifier	(1, 1)	0.99	0.99	0.99	0.99
MLP Classifier	(2, 2)	0.98	0.98	0.98	0.98
MLP Classifier	(3, 3)	0.95	0.95	0.95	0.95

2. Confusion Matrices

For each model and n-gram setting, we also generated a **confusion matrix**, which is a table used to show how many predictions were correct or incorrect.



Explanation:

- The diagonal cells (4712 and 4248) show correct predictions.
- The off-diagonal values show incorrect ones.
- This matrix shows excellent performance with very few mistakes.

Final Selected Model :

- **Random Forest with 1-gram** gave the highest accuracy and perfect classification in many cases.
- Logistic Regression and MLPClassifier also performed well, especially with 1-gram and 2-gram.
- Using 3-gram gave slightly lower results, likely due to more noise and data sparsity.

Therefore, **Random Forest with 1-gram TF-IDF** was selected as the final model and saved for future predictions.

4. Conclusion:

This project focused on building a machine learning system to detect fake news using natural language processing techniques. We collected and combined real and fake news articles, cleaned the text using NLP methods like stopword removal and tokenization, and then converted the text into numerical features using TF-IDF with different n-gram settings. After that, we trained three different machine learning models — Logistic Regression, Random Forest, and MLPClassifier — and evaluated them using accuracy, precision, recall, F1-score, and confusion matrices.

All models performed well, especially when using 1-gram TF-IDF features. Among them, the **Random Forest classifier with 1-gram** performed the best. It achieved the highest accuracy and showed excellent results in the confusion matrix, with very few misclassifications. This is because Random Forest works well with large feature sets and is good at handling complex patterns in the data.

In the future, the project can be improved by using more advanced techniques. We can try deep learning models like LSTM or BERT for even better performance. Also, the model can be trained on a larger dataset, including multilingual news, to increase its accuracy and scope. Finally, integrating the system into a live web application (like with Streamlit) can help users easily check if an article is fake or true.

This project shows how machine learning can be used to solve real-world problems like fake news, helping people access more trustworthy information online.

5.References

1. Scikit-learn Developers. (2023). *Scikit-learn: Machine learning in Python*. https://scikit-learn.org
2. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media. https://www.nltk.org
3. Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. *Computing in Science & Engineering*, 9(3), 90-95. https://matplotlib.org
4. Waskom, M. (2021). *Seaborn: Statistical data visualization*. https://seaborn.pydata.org
5. Streamlit Inc. (2024). *Streamlit: Turn data scripts into shareable web apps*. https://streamlit.io
6. Joblib Developers. (2023). *Joblib: Python serialization and job management*. https://joblib.readthedocs.io
7. Kaggle. (2019). *Fake and real news dataset*. https://www.kaggle.com/clmentbisaillon/fake-and-real-news-dataset
8. Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.